NMIMS · MUMBAI

M.Sc. Data Science

Linear Algebra and Network Science · Group Project

EV CHARGING STATION OPTIMIZATION FOR MUMBAI

A Road-Network, Centrality and Facility-Location Approach

using SVD, Louvain Communities, Greedy Search and Integer Linear Programming

PROJECT REPORT

TEAM MEMBERS

Team Member	Roll No.
Samiksha Shikare	A021
Kshiti Vartak	A015
Rashmi Ashok	A017
Abin Pillai	A029

Contents

1	Introduction
2	Problem Statement and Objectives
3	Dataset and Data Sources
4	System Architecture and Methodology
5	Mathematical Formulation: Where Mathematics Is Used
6	Programming Implementation
7	Results and Interpretation
8	Limitations and Responsible Interpretation
9	Conclusion and Future Work
10	References and Links
A	Appendix A: Running the Project
B	Appendix B: Mapping to the Evaluation Rubric

Abstract

Deciding where to install new electric-vehicle (EV) chargers is a siting problem: a limited number of stations must serve as much charging demand as possible, over real roads, without ignoring existing infrastructure or whole parts of the city.

This project represents Mumbai's road network as a graph of **8,574 nodes and 16,284 edges**, estimates charging demand from **47 point-of-interest categories**, denoises traffic data with the **Singular Value Decomposition**, scores junctions with **four centrality measures**, partitions the network into **43 Louvain communities**, and selects **12 new stations** by solving a **Maximum Coverage Location Problem** with both a greedy heuristic and an exact ILP (CBC solver).

Under the model's assumptions, modeled demand coverage rises from **20.7% to 88%** and the average drive to a charger falls from **6.62 km to 1.36 km**.

1. Introduction

1.1 Background

Electric mobility is growing quickly in Indian cities, but charging infrastructure does not automatically grow in step with it. A driver's confidence in switching to an EV depends heavily on a simple question: 'is there a charger reachable without a long detour?' Mumbai is a particularly interesting test case because its geography (a long, narrow peninsula, creeks, highways and bottleneck junctions) means that two places that look close on a map can be far apart by road.

A charging station is a fixed, expensive investment. Where it is placed determines how many people it can realistically serve for years. Choosing sites by intuition ("near malls", "where population is highest") ignores road connectivity, nearby existing chargers and overlapping service areas. Choosing them with a mathematical model makes the decision transparent, repeatable and defensible.

1.2 Why Linear Algebra and Network Science?

A city's road system is naturally a **network**: junctions are nodes and road segments are edges. Once the city is written as a graph, its structure can be encoded in matrices (adjacency matrix, traffic matrix) and studied with **linear algebra**: eigenvectors reveal which junctions are connected to other important junctions, and the SVD separates meaningful traffic patterns from noise. Network science then supplies the vocabulary (centrality, communities, shortest paths) to describe the city's structure, and optimization turns that structure into a concrete decision.

1.3 What this project does?

> We use Mumbai's road network, estimated EV charging demand, network importance measures and optimization algorithms to find suitable locations for new EV charging stations so that as much modeled demand as possible lies within a short drive of a charger.

1.4 Scope of this report

This report documents the problem statement (Section 2), the data (Section 3), the overall pipeline (Section 4), every piece of mathematics used and exactly where it is used (Section 5), the software implementation (Section 6), the results and their correct interpretation (Section 7), and the limitations of the study (Section 8). The sections follow the evaluation rubric: problem statement and objectives, mathematical formulation, programming implementation, and results and interpretation.

2. Problem Statement and Objectives

2.1 Problem statement

> **Core question**
> If a city wants to build new EV charging stations, where should it put them so that the maximum number of people can easily access a charger?

Concretely, suppose Mumbai is to install **p = 12** new stations. Given the road network, the spatial distribution of estimated demand, the existing stations and a service standard of **R = 2 km**, select 12 sites from a set of candidate junctions that maximize the demand covered, while also reducing the average driving distance to the nearest charger and keeping the solution geographically balanced across communities.

2.2 Why simple rules fail

Simple rule	Why it can fail
"Put chargers near malls"	Malls may already have chargers, may be poorly connected by road, and cover only one pocket of demand.
"Put chargers where population is highest"	High-population nodes can sit close together; their service areas overlap, so the second and third stations add little new coverage.
"Put chargers at the busiest junctions"	Traffic alone ignores whether demand exists nearby and whether an existing charger already serves the area.
"Use straight-line distance"	Drivers follow roads. A river, highway or one-way system can make a 1 km straight-line gap a 2 km or longer drive.

2.3 Objectives

1. **Model** Mumbai's road system as a weighted graph so that distance means driving distance.
2. **Estimate** where charging demand is likely to be high using points of interest (POIs) as a proxy.
3. **Denoise** traffic-related data using low-rank approximation (SVD) to retain the dominant patterns.
4. **Quantify** junction importance using degree, betweenness, closeness and eigenvector centrality combined with demand.
5. **Partition** the network into communities (Louvain) so that coverage can be checked for balance across regions.
6. **Optimize** site selection as a Maximum Coverage Location Problem, solved by a greedy heuristic and compared with an exact ILP.
7. **Evaluate** the solution with coverage and average-drive-distance metrics, before and after the build.

8. **Communicate** the result through an interactive map (map.html) and an analytical report (report.html).

2.4 Expected deliverables

Deliverable	Description
Source code	Python pipeline: network.py, demand.py, linalg.py, centrality.py, communities.py, optimise.py, coverage.py, dashboard.py, run_pipeline.py
map.html	Interactive map: road network, demand, existing stations, recommended sites, service radius, drive-to-charger layers
report.html	Analytical report: traffic, SVD, centrality, communities, coverage curve, optimization and equity results
This report + presentation	Written documentation, followed by demonstration and viva voce

3. Dataset and Data Sources

The study combines real geographic data with modeled analytical inputs. It is important to be precise about which is which, because the interpretation of the results depends on it.

3.1 Dataset summary

Dataset	Source	Used for	Nature of data
Road network	OpenStreetMap (OSM)	Graph of junctions and road segments (network.py)	Real geographic data. 8,574 nodes, 16,284 edges
Points of interest	OpenStreetMap (OSM)	Estimating charging demand (demand.py)	Real geographic data. 47 POI categories
Existing chargers	Input data supplied to the pipeline	Baseline coverage; excluded duplicate siting	Existing station locations used as the "before" scenario
Traffic-related data	Modeled / simulated for the study	SVD denoising and analysis (linalg.py)	Not real-time Mumbai traffic; see Section 8

3.2 Dataset links

Resource	What it provides	Link
OpenStreetMap	Primary map database: roads, junctions, POIs	openstreetmap.org
OSM Overpass API	Query interface for extracting OSM roads and POIs by area and tag	overpass-api.de
OSM copyright / licence	Open Database Licence (ODbL) terms and attribution	openstreetmap.org/copyright
OSM tag: charging station	Standard tag for existing EV charging stations	wiki.openstreetmap.org
OSMnx (reference)	Peer-reviewed methodology for street networks from OSM	github.com/gboeing/osmnx
COIN-OR CBC solver	Open-source ILP solver used for the exact optimization	github.com/coin-or/Cbc

3.3 Preparation of the data

- **Road network:** OSM roads are converted into a graph. Each node is a junction (with coordinates); each edge is a road segment weighted by its length in metres.
- **POIs:** OSM features are grouped into 47 categories (offices, residential areas, hospitals, schools, shopping areas, hotels, restaurants, commercial areas and others). Each POI is attached to its nearest road node.
- **Existing stations:** the locations of already-installed chargers form the baseline against which the new stations are judged.
- **Candidate sites:** unsuitable nodes are filtered out and the rest are ranked, leaving a candidate list much smaller than the 8,574 nodes.

> **Attribution**
> Map data © OpenStreetMap contributors, available under the Open Database Licence (ODbL).

4. System Architecture and Methodology

The project is a multi-stage pipeline. Each stage answers one question and passes its output to the next. The figure below shows the stages and the Python file responsible for each.

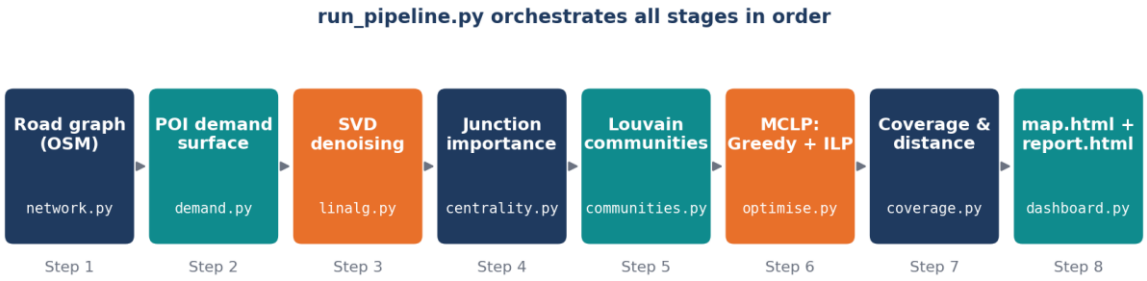


Figure 1. End-to-end pipeline; run_pipeline.py executes the stages in order.

4.1 Stage-by-stage summary

#	Stage	File	Question answered
1	Road network	network.py	Where are the roads and how are they connected?
2	Demand estimation	demand.py	Where is charging demand likely to be high?
3	Linear algebra	linalg.py	Which traffic patterns are signal and which are noise?
4	Centrality	centrality.py	Which junctions are structurally important?
5	Communities	communities.py	Which areas naturally belong together?
6	Optimization	optimise.py	Where should the 12 stations be built?
7	Coverage	coverage.py	How much better is access after the build?
8	Dashboard	dashboard.py	How do we show the result (map and report)?

4.2 Design decisions

- **Driving distance, not straight-line distance.** All coverage decisions use shortest-path distance on the road graph.
- **Several signals, not one.** Demand, centrality, connectivity and existing infrastructure are combined, because any single measure can mislead (high traffic with no demand; high demand next to an existing charger).
- **Heuristic and exact solver together.** The greedy method is fast; the ILP can prove optimality within the model. Comparing them validates the heuristic.
- **Equity check.** Community-level coverage is reported so that the solution is not concentrated in one region.

5. Mathematical Formulation: Where Mathematics Is Used

This section states, for every stage of the pipeline, the mathematical object used, the formulas, and the exact purpose in the project. Table 5.0 gives the overview; the subsections give the detail.

5.0 Overview: concept → mathematics → location in the project

Concept	Mathematics	File	Purpose in the project
Road graph	Weighted graph G=(V,E,w); adjacency matrix A; Dijkstra shortest paths	network.py	Represent Mumbai; compute driving distances
Demand surface	Weighted sum of POI counts; min–max normalisation	demand.py	Estimate where chargers are needed
Traffic denoising	SVD, low-rank approximation, Eckart–Young theorem, eigenvalues of T^TT, Gram–Schmidt / QR	linalg.py	Keep dominant patterns, remove noise
Junction importance	Degree, betweenness, closeness, eigenvector centrality ($Ax = \lambda x$, power iteration)	centrality.py	Rank junctions structurally
Priority score	Convex combination of normalised features	centrality.py / demand.py	Combine demand and network importance
Communities	Modularity Q maximisation (Louvain)	communities.py	Group the network; check equity
Site selection	Maximum Coverage Location Problem: 0–1 ILP; greedy submodular maximisation	optimise.py	Choose 12 stations
Evaluation	Weighted coverage ratio; weighted mean of shortest-path distance	coverage.py	Measure before / after improvement

Table 5.0. Where each piece of mathematics enters the project.

5.1 Graph model of the road network

Mumbai's roads are modeled as a weighted, undirected graph G = (V, E, w) with n = |V| = 8,574 junction nodes and m = |E| = 16,284 road segments. The weight w(e) of an edge is the length of that segment in metres. The structure is stored in the **adjacency matrix** $A \in \mathbb{R}^{n \times n}$:

$$A_{uv} = 1 \text{ if } (u, v) \in E, \text{ else } 0 \quad (5.1)$$

The **driving distance** between two nodes is the length of the shortest path, computed with Dijkstra's algorithm:

$$d(u, v) = \min \text{ over paths } P \text{ from } u \text{ to } v \text{ of } \Sigma_{e \in P}\, w(e) \quad (5.2)$$

Used for: deciding whether a demand node lies within the 2 km service standard of a station, and for the average-drive-distance metric.

5.2 Demand estimation

Each POI belongs to one of K = 47 categories. For a road node v, let $n_k(v)$ be the number of POIs of category k assigned to v (or its neighbourhood) and let $\omega_k \geq 0$ be the weight given to category k. The raw demand is a weighted sum, then normalised to [0, 1]:

$$D(v) = \Sigma_{k=1..K}\, \omega_k \cdot n_k(v) \qquad \tilde{D}(v) = (D(v) - D_{min}) \,/\, (D_{max} - D_{min}) \quad (5.3)$$

Used for: the demand weight w_i of each demand node in the coverage objective and the heat-map layer "Charging demand" on the map.

5.3 Linear algebra: SVD denoising of traffic data

Let $T \in \mathbb{R}^{r \times c}$ be the traffic data matrix. Its **Singular Value Decomposition** is

$$T = U\, \Sigma\, V^T = \Sigma_{i=1..p}\, \sigma_i \cdot u_i\, v_i^T, \quad \sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_p \geq 0 \quad (5.4)$$

where U and V have orthonormal columns and Σ is diagonal. The singular values are linked to eigenvalues: the σ_i^2 are the eigenvalues of T^TT, and v_i are its eigenvectors:

$$T^T T\, v_i = \sigma_i^{\,2}\, v_i \quad (5.5)$$

Large singular values correspond to dominant, systematic traffic patterns; small ones mostly capture random variation. The rank-k approximation keeps only the first k terms:

$$T_k = \Sigma_{i=1..k}\, \sigma_i\, u_i\, v_i^T \quad (5.6)$$

By the **Eckart–Young theorem**, T_k is the best rank-k approximation of T in the Frobenius norm, and its error is exactly determined by the discarded singular values; the retained information is measured by an energy ratio:

$$\| T - T_k \|_F = \sqrt{(\, \Sigma_{i>k}\, \sigma_i^{\,2}\,)} \qquad E(k) = \Sigma_{i \leq k}\, \sigma_i^{\,2} \,/\, \Sigma_i\, \sigma_i^{\,2} \quad (5.7)$$

Gram–Schmidt orthogonalisation (the QR factorisation) is the constructive way to build an orthonormal basis from a set of vectors $a_1, \ldots, a_k$, and underlies the orthonormal structure used in the SVD and eigenvector computations:

$$q_j = (\, a_j - \Sigma_{i<j}\, \langle a_j, q_i \rangle\, q_i \,) \,/\, \| \, a_j - \Sigma_{i<j}\, \langle a_j, q_i \rangle\, q_i \, \| \quad (5.8)$$

Used for: linalg.py, which keeps the important components and removes weaker, noisy ones; results are shown in the SVD section of report.html. This also demonstrates matrix operations, eigenvectors, SVD and Gram–Schmidt from the course syllabus.

5.4 Network science: centrality measures

Centrality assigns each node a number describing how important it is within the network. Four measures are used.

Measure	Formula	Meaning for a junction
Degree	$C_D(v) = \deg(v) / (n - 1)$	How many roads meet here
Betweenness	$C_B(v) = \Sigma_{s \neq v \neq t} \sigma_{st}(v) / \sigma_{st}$	How often it lies on shortest routes between other places (σ_{st} = number of shortest s–t paths; $\sigma_{st}(v)$ = those passing through v)
Closeness	$C_C(v) = (n - 1) / \Sigma_{u \neq v} d(v, u)$	How quickly it reaches all other junctions
Eigenvector	$A\,x = \lambda_1 x$, $x_v = (1/\lambda_1) \Sigma_{u \sim v} x_u$	Whether it is connected to other important junctions

The **eigenvector centrality** vector x is the eigenvector of the adjacency matrix A for its largest eigenvalue λ_1. By the Perron–Frobenius theorem, for a connected graph this eigenvector has all positive entries, and it can be computed by **power iteration**:

$$x^{(t+1)} = A\,x^{(t)} \,/\, \| A\,x^{(t)} \| \quad (5.9)$$

Used for: centrality.py. A node with high eigenvector centrality is not merely well connected; its neighbours are themselves important.

5.5 Siting priority score

Demand and network importance are combined into one **siting priority score** using min–max normalised features (denoted with a tilde) and non-negative weights that sum to one:

$$S(v) = \alpha_1 \tilde{D}(v) + \alpha_2 \tilde{C}_D(v) + \alpha_3 \tilde{C}_B(v) + \alpha_4 \tilde{C}_C(v) + \alpha_5 \tilde{C}_E(v), \quad \Sigma \alpha_j = 1,\ \alpha_j \geq 0$$

(5.10)

Used for: ranking nodes into the candidate set and for the "Siting Priority Score" layer on the map. The weights α_j are the values configured in the project code.

5.6 Community detection (Louvain)

A community is a group of nodes more densely connected to each other than to the rest of the network. The Louvain algorithm greedily maximises **modularity**:

$$Q = (1 / 2m) \Sigma_{i,j} [\, A_{ij} - (k_i\, k_j) / (2m) \,] \, \delta(c_i, c_j) \quad (5.11)$$

Here k_i is the degree of node i, m the number of edges, c_i the community of node i, and $\delta(c_i, c_j) = 1$ if i and j are in the same community, else 0. A high Q means the partition has many more internal edges than expected in a random network with the same degrees.

The algorithm repeatedly (i) moves nodes to the neighbouring community with the largest modularity gain ΔQ and (ii) collapses each community into a super-node, until Q cannot be improved. The project finds approximately **43 communities**.

Used for: communities.py, and the equity analysis that reports coverage per community so that the solution does not concentrate in one region.

5.7 Facility-location optimization: the Maximum Coverage Location Problem

Let I be the set of demand nodes (each with weight $w_i \geq 0$ from Eq. 5.3), J the set of candidate sites, R = 2,000 m the service distance, and p = 12 the number of stations to build. Define the set of candidate sites that can serve demand node i:

$$N_i = \{ j \in J : d(i, j) \leq R \} \quad (5.12)$$

Using binary decision variables $x_j = 1$ if a station is built at j, and $y_i = 1$ if demand node i is covered, the MCLP is the **0–1 Integer Linear Program**:

$$\text{maximise} \quad \Sigma_{i \in I} w_i y_i \quad (5.13)$$

$$\text{subject to} \quad y_i \leq \Sigma_{j \in N_i} x_j \quad \text{for all } i \in I \quad (5.14)$$

$$\Sigma_{j \in J} x_j = p \quad (5.15)$$

$$x_j \in \{0, 1\}, \quad y_i \in \{0, 1\} \quad (5.16)$$

Constraint 5.14 says a demand node can only count as covered if at least one selected station lies within R of it; 5.15 limits the budget to p stations. Existing stations are treated as already-open facilities, so the coverage achieved by the final network includes both existing and newly selected sites. The ILP is solved with the **CBC** branch-and-cut solver. When CBC terminates with the optimality gap closed, no feasible selection of p sites has a larger objective value; this is optimality **with respect to the model, objective, constraints and data supplied**, not a guarantee about the real world.

Greedy heuristic

The greedy algorithm starts with an empty selection S and repeatedly adds the candidate with the largest **marginal gain**, i.e. the weight of newly covered demand:

$$j^* = \text{argmax over } j \notin S \text{ of } \Sigma_{i \in N^{-1}(j) \setminus \text{Covered}(S)} w_i \quad (5.17)$$

Because the coverage function is monotone and submodular, the greedy method is guaranteed to reach at least a (1 − 1/e) ≈ 63.2% fraction of the optimal value (Nemhauser, Wolsey and Fisher, 1978); in practice it is usually much closer. It is very fast, which is why it is compared with the exact ILP.

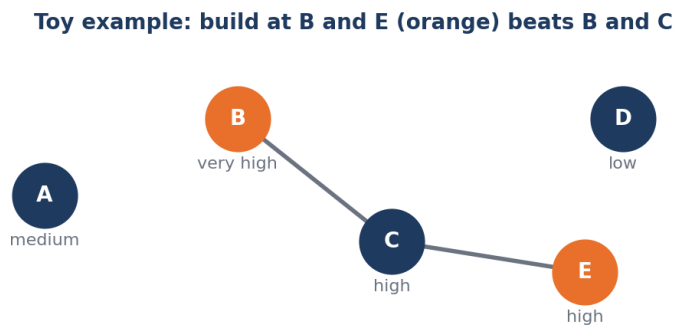


Figure 2. Toy example: B covers C, so adding C gives little new coverage; E reaches demand B cannot. The optimizer picks B + E rather than B + C.

Used for: optimise.py. This is the central decision step of the project.

5.8 Evaluation metrics

Let F = (existing stations) ∪ S be the final set of stations. The two main metrics are the **weighted coverage** and the **weighted average drive distance**:

$$\text{Coverage}(F) = 100 \cdot \Sigma_{i \in I(F)} w_i \;/\; \Sigma_{i \in I} w_i \quad (\%) \qquad (5.18)$$

$$\bar{d}(F) = \Sigma_{i \in I} w_i \cdot d(i, F) \;/\; \Sigma_{i \in I} w_i \quad (\text{km}), \quad d(i, F) = \min_{f \in F} d(i, f) \qquad (5.19)$$

Here I(F) is the set of demand nodes whose nearest station in F is within distance R, and d(i, F) is the driving distance from node i to its nearest station.

The same coverage ratio computed over the nodes of community c gives an **equity measure** $\text{Coverage}_c(F)$, which shows whether some communities are served much worse than others.

Used for: coverage.py and the "Drive to charger: today" and "after build" layers on the map.

6. Programming Implementation

The project is implemented in Python as a modular pipeline. Each file has one responsibility; run_pipeline.py runs everything in the correct order and writes the two outputs.

6.1 Code structure

File	Responsibility	Mathematics implemented
network.py	Builds the road graph from OSM	Graph G, adjacency, shortest paths
demand.py	Builds the demand surface from 47 POI categories	Weighted sums, normalisation
linalg.py	Linear-algebra analysis of traffic data	SVD, low-rank approximation, eigen-analysis, Gram–Schmidt
centrality.py	Junction importance scores	Degree, betweenness, closeness, eigenvector centrality
communities.py	Network communities	Louvain / modularity
optimise.py	Selects the 12 stations	MCLP: greedy and ILP (CBC)
coverage.py	Measures solution quality	Coverage ratio, mean drive distance, equity
dashboard.py	Generates map.html and report.html	Visualisation of all results
run_pipeline.py	Orchestrates all stages	-

6.2 How to run

Three run modes are available. They differ in how much is computed and displayed, not in the mathematical goal.

Command	Mode	Use
run_pipeline.py	Full run	Final results; complete configured analysis
run_pipeline.py --quick	Quick run	Smaller, faster test while developing
run_pipeline.py -v	Verbose	Shows progress messages; for debugging
run_pipeline.py --quick -v	Quick + verbose	Fast debugging with detailed logs

```
.\.venv\Scripts\python.exe run_pipeline.py
.\.venv\Scripts\python.exe run_pipeline.py --quick
.\.venv\Scripts\python.exe run_pipeline.py -v
```

6.3 Pipeline outputs

- **map.html** (interactive): layers for road network, charging demand, siting priority score, existing stations, recommended sites, 2 km service radius, and drive-to-charger today / after build.
- **report.html** (analytical): traffic analysis, SVD analysis, centrality, communities, coverage-vs-stations curve, optimization results and equity by community.

6.4 Reading the map correctly

Map element	Interpretation
Orange numbered markers	The 12 recommended sites chosen by the optimizer. The numbers are identifiers, not a ranking, unless the code states otherwise.
Existing stations	Chargers already present in the input data (the baseline).
Charging demand	Modeled demand estimated from POIs; denser or darker areas mean more modeled demand.
Siting priority score	Pre-optimization attractiveness of locations under the scoring system.
Service radius (2 km)	A visual aid for the service standard. The mathematics uses road-network distance; a circle is not a driving route.
Drive to charger: today / after build	Distance demand must travel to the nearest existing charger, before and after adding the recommended stations.

7. Results and Interpretation

7.1 Headline results

The optimization selected **12 new charging stations**. Compared with the situation using existing chargers only, the model reports:

Metric	Before	After	Change
Modeled demand covered (within 2 km)	20.7%	88%	+67.3 percentage points
Average drive distance to a charger	6.62 km	1.36 km	−5.26 km (about 79.5% shorter)
Number of new stations	-	12	-

Table 7.1. Before / after comparison. Differences are computed from the reported values.

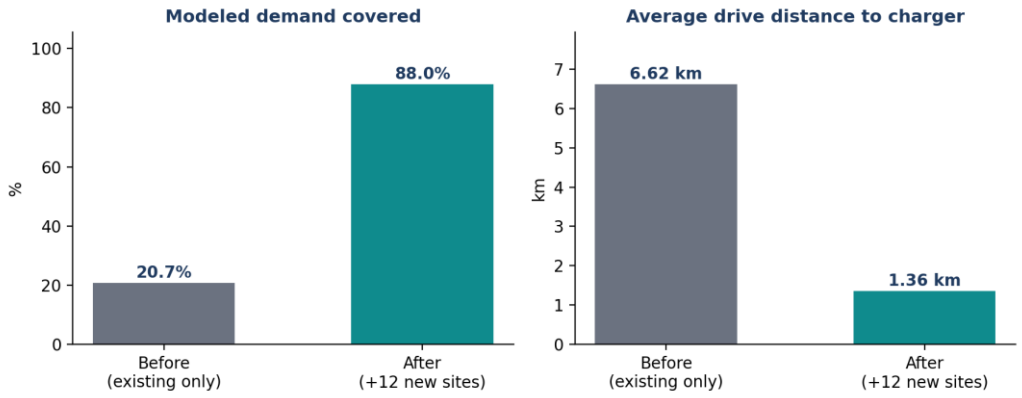


Figure 3. Coverage rises sharply and average drive distance falls under the model's assumptions.

7.2 Interpretation

- **Coverage.** Before the build, only about one-fifth (20.7%) of modeled demand is within the service distance of a charger. With 12 well-chosen sites, this rises to 88%. The gain per station is large because the optimizer avoids overlapping service areas.
- **Drive distance.** The average drive falls from 6.62 km to 1.36 km. For a driver, this is the difference between a planned detour and a convenient nearby stop.
- **Diminishing returns.** The coverage-versus-stations curve in report.html (0 stations → 4 → 8 → 12) shows coverage improving as stations are added, with later stations typically adding less than early ones. This is the expected behaviour of a submodular objective.
- **Greedy vs ILP.** The fast greedy heuristic is compared with the exact ILP; agreement between the two supports the quality of the heuristic, while the ILP certifies optimality within the model.
- **Equity.** Community-level results indicate whether different parts of the network receive reasonable access, so the solution is not simply stacked in one dense region.

> **How to state the result**
>
> "Our model estimates that approximately **88% of the modeled charging demand** is within the defined service distance of a charging station, compared with 20.7% today, and that the average driving distance falls from 6.62 km to 1.36 km under the project's assumptions."
>
> Do **not** say "88% of Mumbai's population is covered" unless population is the demand variable in the code.

7.3 What the map does and does not prove

The map does not by itself prove that the selected locations are optimal. It visualises the model's selected locations, demand, existing infrastructure and coverage so that the optimization result can be inspected geographically. Optimality is established by the ILP in the mathematical model; the map makes the outcome understandable.

7.4 Why optimization beats "highest demand first"

The highest-demand locations can lie close together, so their service areas overlap heavily. A location with slightly lower individual demand may add far more new coverage. The optimizer evaluates the combined effect of the whole selection, which a ranking by demand cannot do (Figure 2).

8. Limitations and Responsible Interpretation

Limitation	Consequence / how to phrase it
Traffic data are modeled or simulated	Say: "The road network is based on real geographic data, while some analytical/traffic inputs are modeled or simulated for the study." Do not claim real-time Mumbai traffic was collected.
Demand is a proxy	POI-based demand estimates where charging is likely needed; it is not measured charging sessions or EV ownership.
Coverage is not population	88% refers to the project's defined demand measure within the service threshold, not 88% of residents.
Optimality is model-relative	"Optimal" means no better solution exists under this objective, these constraints and this data.
Service radius vs road distance	The 2 km circle on the map is a visual aid; the calculation uses road-network distance.
Real deployment constraints	Land availability, grid capacity, installation cost, permits, charger type, parking and actual EV demand are not modeled.

Because of these points, the system is best described as a **decision-support and optimization system for identifying promising EV charging locations**, not as a guarantee that real-world deployment will reproduce exactly the same numbers.

9. Conclusion and Future Work

9.1 Conclusion

This project shows how Linear Algebra and Network Science can be combined with optimization to solve a practical urban-planning problem. A real road network was converted into a graph; demand was estimated from points of interest; traffic data were denoised with SVD; junctions were scored by four centrality measures; the network was partitioned into about 43 communities; and 12 new stations were selected by a Maximum Coverage Location Problem solved with both a greedy heuristic and an exact ILP. Under the model's assumptions, coverage of modeled demand increases from 20.7% to 88% and average drive distance falls from 6.62 km to 1.36 km.

The main strength of the work is not a new algorithm but the **integration of several established techniques into one transparent decision pipeline**, with an interactive map and analytical report to support interpretation.

9.2 Future work

- Replace modeled traffic inputs with measured traffic or mobility data.
- Use population, vehicle-registration or charging-session data to calibrate demand.
- Add capacity constraints (chargers per site, queueing) and cost or grid-capacity constraints to the ILP.
- Study sensitivity to the service distance R and the number of stations p.
- Extend to other cities and compare results across road-network structures.

10. References and Links

10.1 Academic references

[1] Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139.

[2] Church, R., & ReVelle, C. (1974). The maximal covering location problem. *Papers of the Regional Science Association*, 32, 101–118.

[3] Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14, 265–294.

[4] Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008.

[5] Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

[6] Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2), 163–177.

[7] Newman, M. E. J. (2018). *Networks* (2nd ed.). Oxford University Press.

[8] Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.

10.2 Data and software links

[9] OpenStreetMap: https://www.openstreetmap.org

[10] OpenStreetMap licence (ODbL): https://www.openstreetmap.org/copyright

[11] Overpass API: https://overpass-api.de

[12] OSM tag amenity=charging_station: https://wiki.openstreetmap.org/wiki/Tag:amenity%3Dcharging_station

[13] OSMnx: https://github.com/gboeing/osmnx

[14] COIN-OR CBC solver: https://github.com/coin-or/Cbc

Appendix A: Running the Project

1. Open a terminal in the project folder.
2. Activate or use the project's virtual environment (.venv).
3. Run a quick test first to confirm everything works.

```
.\.venv\Scripts\python.exe run_pipeline.py --quick -v
```

4. Run the full pipeline for final results.

```
.\.venv\Scripts\python.exe run_pipeline.py
```

5. Open map.html in a browser to explore the map and report.html for the analysis.

> Our project is an EV charging station placement optimization system. We take a real road network, estimate charging demand from points of interest and other inputs, analyze important road junctions and network communities, generate suitable candidate locations, and then use optimization to select a limited number of charging stations. Finally, we calculate coverage and travel distance and visualize the results through an interactive map and analytical report.

Appendix B: Mapping to the Evaluation Rubric

Rubric criterion	Marks	Where addressed in this report
Problem Statement and Objectives	2	Section 2 (problem, why simple rules fail, 8 objectives, deliverables); Section 1
Mathematical Formulation	2	Section 5 (graph model, demand, SVD, Gram–Schmidt, centralities, modularity, MCLP ILP, greedy bound, metrics)
Programming Implementation	1	Section 6 and Section 4 (module structure, run modes, outputs); Section 3 (data)
Results and Interpretation	2.5	Section 7 (before/after, interpretation, what the map proves) and Section 8 (limitations)
Total	7.5	